DỰ BÁO RỦI RO VÀ CƠ CHẾ KÝ QUỸ ĐỘNG CHO HỢP ĐỒNG TƯƠNG LAI VN30F1M BẰNG MÔ HÌNH EGARCH-X

Tài liệu 2: Mã nguồn và thuật toán

Nguyễn Hoài Nam Nguyễn Thuận Hiếu
Đặng Chí Hiếu

Khoa Toán – Thống kê, Trường Đại học Tôn Đức Thắng

Tháng 10 năm 2026

Tóm tắt nội dung

Tài liệu mô tả cách các phương pháp ở Tài liệu 1 được cài đặt thành mã nguồn Python có khả năng tái lập. Nội dung gồm kiến trúc mã, các thuật toán lõi (đệ quy phương sai, hàm hợp lý, ước lượng hợp lý cực đại, Bayes biến phân, NUTS, mô phỏng rủi ro nhiều phiên, quy tắc ký quỹ năm bước) và hạ tầng bảo đảm tính đúng đắn của quy trình tính toán dài hạn (băm đầu vào, điểm kiểm tra ba tầng, ghi nguyên tử, phân mảnh, cổng chuyển giai đoạn, khóa khối kiểm tra). Tài liệu chỉ mô tả những gì mã thực sự làm; những điểm thiết kế còn là lựa chọn tạm thời được nêu ở Mục 5.

Từ khóa: Cài đặt, vi phân tự động, JAX, NumPyro, điểm kiểm tra, khả năng tái lập.

Mục lục

1 Kiến trúc mã nguồn **3**

1.1 Nguyên tắc thiết kế . . . 3
1.2 Cấu trúc thư mục . . . 3
1.3 Ánh xạ từ lý thuyết sang mô-đun . . . 3
1.4 Nền tảng tính toán . . . 3

2 Thuật toán lõi **4**

2.1 Dữ liệu và xử lý đáo hạn . . . 4
2.2 Đệ quy phương sai và hàm hợp lý . . . 4
2.3 Đổi biến và hậu nghiệm . . . 5
2.4 Ước lượng hợp lý cực đại . . . 5
2.5 Mô hình nền GARCH(1,1) . . . 6
2.6 Bayes biến phân . . . 6
2.7 NUTS . . . 7
2.8 Dự báo một phiên . . . 8
2.9 Mô phỏng rủi ro nhiều phiên . . . 8
2.10 Tổn thất thực tế và các thước đo đối chứng . . . 9
2.11 Quy tắc ký quỹ năm bước . . . 9
2.12 Chỉ tiêu và kiểm định . . . 9

3 Hạ tầng bảo đảm tính đúng đắn **10**

3.1 Bước, đơn vị và hồ sơ . . . 10
3.2 Mã băm đầu vào . . . 10
3.3 Điểm kiểm tra ba tầng . . . 10
3.4 Tính tất định và hạt giống . . . 11
3.5 Phân mảnh và song song hóa . . . 11
3.6 Cổng chuyển giai đoạn và khóa khối kiểm tra . . . 11
3.7 Đồng bộ và vòng lặp điều phối . . . 11

4 Kiểm thử và kiểm chứng cài đặt **12**

5 Những điểm cần lưu ý khi diễn giải **12**

1 Kiến trúc mã nguồn

1.1 Nguyên tắc thiết kế

Mã nguồn tuân theo bốn nguyên tắc.

1. **Một nguồn sự thật cho hàm hợp lý.** Hợp lý cực đại, Bayes biến phân và NUTS dùng chung một hàm logarit hợp lý viết bằng JAX. Nhờ vậy sự khác biệt giữa các phương pháp chỉ đến từ thuật toán suy diễn, không đến từ khác biệt cài đặt.
2. **Một nguồn sự thật cho tham số.** Mọi giá trị điều chỉnh được (cỡ khối, số lần lặp, siêu tham số ký quỹ, ngưỡng cổng) nằm trong một tệp cấu hình duy nhất. Mã không chứa hằng số ngầm.
3. **Mỗi bước là một đơn vị độc lập có thể dừng và chạy tiếp.** Kết quả trung gian được ghi nguyên tử, và mã băm của đầu vào quyết định kết quả cũ còn dùng được hay không.
4. **Không rò rỉ thông tin tương lai.** Mọi thống kê chuẩn hóa chỉ dùng dữ liệu đến thời điểm ước lượng lại; khối kiểm tra bị che khỏi mọi bước cho tới khi cấu hình được khóa.

1.2 Cấu trúc thư mục

Bảng 1: Cấu trúc thư mục của kho mã

Thư mục	Nội dung
`cau_hinh/`	`de_cuong.toml` (mọi tham số), `the_che.toml` (thông số thể chế kèm nguồn và ngày xác minh)
`ma_nguon/de_cuong/`	Thư viện: Mô hình, phân phối, ước lượng, mô phỏng, đánh giá, ký quỹ, khung điều phối
`quy_trinh/`	Các bước b00 đến b15 và điểm vào `chay.py`
`ket_qua/`	Đầu ra theo từng bước, hồ sơ, điểm kiểm tra, nhật ký
`kiem_thu/`	Kiểm thử tự động (23 kiểm thử)
`du_lieu/`	Dữ liệu gốc và chuỗi ký quỹ lịch sử của tổ chức bù trừ

1.3 Ánh xạ từ lý thuyết sang mô-đun

1.4 Nền tảng tính toán

Thư viện JAX cung cấp ba công cụ mà toàn bộ cài đặt dựa vào. Thứ nhất, *vi phân tự động*: Gradient và Hessian của hàm hợp lý được tính chính xác (không dùng sai phân hữu hạn), điều kiện cần cho L-BFGS ổn định và cho NUTS. Thứ hai, *biên dịch đúng lúc* (`jit`): Hàm được biên dịch một lần rồi chạy ở tốc độ mã máy. Thứ ba, *vòng lặp có cấu trúc* (`lax.scan`): Đệ quy phương

Bảng 2: Ánh xạ giữa nội dung lý thuyết (Tài liệu 1) và mô-đun cài đặt

Nội dung	**Mục ở Tài liệu 1**	**Mô-đun**
Dữ liệu, biến, biến lịch	Mục 2	`du_lieu.py`
Phân phối sai số	Mục 3.6	`phan_phoi.py` (NumPy), `mo_hinh_jax.py` (JAX)
EGARCH-X, hàm hợp lý, đổi biến	Mục 3, 4.1	`mo_hinh_jax.py`
GARCH(1,1) nền	Mục 3.1	`garch_nen.py`
MLE, VB, NUTS, dự báo	Mục 4	`uoc_luong.py`
Mô phỏng khôi phục, SBC	Mục 5	`mo_phong.py`
Chỉ tiêu, kiểm định, so sánh	Mục 6	`danh_gia.py`
Mô phỏng H phiên, ký quỹ	Mục 7	`ky_quy.py`
Điều phối, băm, điểm kiểm tra	(hạ tầng)	`khung.py`, `dong_bo.py`

sai theo thời gian được viết dưới dạng quét tuần tự mà vẫn khả vi. Độ chính xác kép được bật toàn cục. NumPyro cung cấp suy diễn biến phân ngẫu nhiên (`SVI`) và NUTS trên cùng mật độ.

Để JAX chỉ biên dịch một lần cho mọi mốc ước lượng lại, mọi mảng có độ dài cố định bằng cả bảng ngày; cửa sổ ước lượng được chọn bằng một mặt nạ logic thay vì cắt mảng.

2 Thuật toán lõi

2.1 Dữ liệu và xử lý đáo hạn

Hàm `xay_ngay` trong `du_lieu.py` gộp nến phút của F1M thành bảng ngày, xác định ngày đáo hạn (thứ Năm thứ ba, lùi về phiên trước nếu nghỉ), cờ phiên chuyển J_t, số phiên đến đáo hạn τ_t và ba biến giả lịch. Cách xử lý phiên chuyển do khóa cấu hình `xu_ly_dao_han` quyết định. Với giá trị `"f2m"` (cấu hình chính thức), hàm đọc tệp giá ngày VN30F2M (`doc_f2m`), lấy giá đóng cửa F2M của phiên liền trước mỗi phiên chuyển và thay r_t, r_t^d bằng lợi suất cùng hợp đồng; với giá trị `"loai"`, lợi suất phiên chuyển để khuyết như cách cũ. Bảng ngày có hai cờ riêng biệt: `J` là phiên chuyển theo lịch (dùng cho biến giả và nhóm phiên), `khuyet` là lợi suất khuyết χ_t (dùng cho hàm hợp lý và đệ quy). Hàm `dac_trung` trả cờ khuyết cho phần ước lượng. Mã băm dữ liệu gồm cả tệp F2M, nên đổi dữ liệu này làm mọi bước chạy lại.

2.2 Đệ quy phương sai và hàm hợp lý

Hàm `loc` trong `mo_hinh_jax.py` thực hiện đệ quy (9) của Tài liệu 1 bằng `lax.scan`. Trạng thái mang theo gồm $(\ln h_{t-1}, z_{t-1}, \chi_{t-1}, \text{cờ đầu chuỗi})$.

Thuật toán 1. Đệ quy lọc $\ln h_t$ và z_t của EGARCH-X

Đầu vào: Tham số $\theta = (\mu, \omega, \kappa_1, \kappa_2, \beta, \gamma, \delta, \theta_{\mathscr{E}})$; chuỗi r_t, cờ lợi suất khuyết χ_t, ma trận $Z_t = [X_{t-1}, D_t]$, $\ln h_1$ khởi tạo ($\ln h_0^*$).
Đầu ra: Chuỗi $\ln h_t$ và z_t cho mọi phiên.

1. Tính $\mu_{|z|} = \mathrm{E}|z|$ từ tham số hình dạng.
2. Với phiên đầu: $\ln h_1 \leftarrow \ln h_0^*$.
3. Với mỗi phiên $t \geq 2$:
   (a) Nếu $\chi_{t-1} = 1$ thì $s \leftarrow 0$; ngược lại $s \leftarrow \kappa_1(|z_{t-1}| - \mu_{|z|}) + \kappa_2 z_{t-1}$.
   (b) $\ln h_t \leftarrow \mathrm{clip}\left(\omega + s + \beta \ln h_{t-1} + Z_t^{\top}(\gamma, \delta), -20, 20\right)$.
4. $z_t \leftarrow 0$ nếu $\chi_t = 1$; ngược lại $z_t \leftarrow (r_t - \mu)\, e^{-\ln h_t/2}$.

Logarit hàm hợp lý (18) là tổng, trên các phiên nằm trong mặt nạ cửa sổ ước lượng, của $\ln f_{\mathscr{E}}(z_t) - \frac{1}{2}\ln h_t$; các phiên $\chi_t = 1$ có đóng góp bằng 0. Hàm `ll_tung_phien` trả về đóng góp từng phiên, từ đó suy ra tổng, véc-tơ điểm số (qua `jacfwd`) và Hessian (qua `hessian`) cho sai số chuẩn.

2.3 Đổi biến và hậu nghiệm

Hàm g chuyển $\xi \to \theta$ theo (20), hàm `g_nghich` làm ngược lại và `log_jacobian` cộng các số hạng $\ln|dg_i/d\xi_i|$ của (24). Mật độ hậu nghiệm không chuẩn hóa trên ξ là

$$\ln \tilde{p}(\xi \mid y) = \ell(g(\xi)) + \ln p(g(\xi)) + \ln|J_g(\xi)|. \tag{1}$$

Đây là hàm duy nhất được truyền cho NumPyro, dưới dạng `numpyro.factor` trên một biến `xi` có tiên nghiệm phẳng; do đó VB và NUTS cùng mục tiêu. Số hạng $\ln p(\theta)$ của β và λ được tính trực tiếp theo ξ để ổn định khi $\sigma(\xi)$ làm tròn thành 0 hoặc 1.

2.4 Ước lượng hợp lý cực đại

Thuật toán 2. MLE nhiều điểm khởi tạo (hàm `mle`)

Đầu vào: Dữ liệu y, đặc tả $(\mathscr{E}, X)$, số điểm khởi tạo $n_0 = 20$, độ tán 0,3, nghiệm ấm (tùy chọn).
Đầu ra: $\hat{\theta}$, $\ell(\hat{\theta})$, số điểm khởi tạo hội tụ về nghiệm tốt nhất, độ tán của ℓ.

1. Giải mô hình đơn giản nhất (phân phối chuẩn, $\gamma = 0$) từ 5 giá trị khởi tạo của $\beta \in \{0{,}5; 0{,}8; 0{,}9; 0{,}95; 0{,}98\}$; giữ nghiệm tốt nhất $\theta_{\text{lồng}}$.
2. Mở rộng $\theta_{\text{lồng}}$ thành điểm khởi tạo của cấu hình đầy đủ ($\gamma = 0$; hình dạng $\nu = 8$ và $\lambda = 0$, riêng GED $\nu = 1{,}5$) rồi chuyển sang ξ.
3. Lập danh sách n_0 điểm: Điểm mở rộng và $n_0 - 1$ điểm cộng nhiễu $N(0, 0{,}3^2)$; nếu có nghiệm

ấm của lần ước lượng trước thì thêm vào đầu danh sách.

4. Với mỗi điểm, cực tiểu hóa $-\ell(g(\xi))$ bằng L-BFGS-B với gradient từ vi phân tự động (dung sai gradient 10^{-6}, tối đa 3.000 vòng). Điểm cho giá trị không hữu hạn bị loại.
5. Chọn nghiệm có ℓ lớn nhất; đếm số điểm có ℓ sai khác không quá 10^{-3} so với nghiệm tốt nhất.

Số điểm hội tụ về cùng nghiệm là chẩn đoán trực tiếp độ khó của bề mặt hợp lý: Nếu hầu hết điểm cho cùng một nghiệm, bề mặt gần như một đỉnh. Khi ước lượng lại theo mốc, nghiệm ấm của mốc trước được thêm vào danh sách nhằm tăng tốc và giữ tính liên tục.

Sai số chuẩn dạng bánh kẹp được tính bằng ma trận quan sát $\hat{A} = -\sum \nabla^2 \ell_t$ (Hessian đầy đủ qua `hessian`) và $\hat{B} = S^\top S$ với S là ma trận $n \times d$ của điểm số từng phiên (qua `jacfwd`); phương sai là $\hat{A}^{-1}\hat{B}\hat{A}^{-1}$. Nếu $\hat{A}$ suy biến thì dùng giả nghịch đảo. Kiểm tra khả nghịch lọc hai đường $\ln h_t$ từ $\ln h_1$ và $\ln h_1 + 2$ rồi báo số phiên để chênh lệch dưới 10^{-6}.

2.5 Mô hình nền GARCH(1,1)

Mô-đun `garch_nen.py` cài đặt (8) của Tài liệu 1 với cùng giao diện dữ liệu và cùng mật độ sai số như EGARCH-X. Đệ quy dùng h_t thay vì $\ln h_t$ và chặn trong $[e^{-20}, e^{20}]$. Tại phiên khuyết, trạng thái "bình phương phần dư" được thay bằng h_t (kỳ vọng của nó). Tham số không ràng buộc là $\xi = (\mu, \ln\omega, \mathrm{logit}(a+\beta), \mathrm{logit}(a/(a+\beta)), \dots)$. Ước lượng dùng cùng thủ tục đa điểm khởi tạo với lưới $(a,\beta) \in \{(0{,}05;0{,}90), (0{,}10;0{,}80), (0{,}03;0{,}95), (0{,}15;0{,}70), (0{,}08;0{,}88)\}$. Mô-đun được kiểm thử bằng cách khôi phục tham số mô phỏng và đối chiếu với gói `arch` (chênh lệch tham số dưới 0,02 và log-hợp lý dưới 3 đơn vị; phần dư khác biệt do cách gói `arch` khởi tạo phương sai đầu).

2.6 Bayes biến phân

Thuật toán 3. Biến phân mean-field và full-rank (hàm `vb`)

Đầu vào: Mục tiêu $\ln \tilde{p}(\xi \mid y)$, điểm khởi tạo ξ_0, họ $\in \{\mathrm{MF}, \mathrm{FR}\}$, số bước 20.000, tốc độ học 0,005, 1.000 mẫu sau cùng.
Đầu ra: Mẫu $\theta^{(s)}$, ELBO cuối, $\hat{k}$.

1. Khởi tạo $q = N(\xi_0,\, 0{,}05^2 I)$ cho cả hai họ (`AutoNormal` cho MF, `AutoMultivariateNormal` cho FR).
2. Lặp 20.000 bước (chia thành đoạn 2.000 bước, mỗi đoạn lưu trạng thái tối ưu): Rút một mẫu $\zeta \sim N(0, I)$, tính $\xi = \mu_q + R\zeta$ (với mean-field, R là ma trận chéo $\mathrm{diag}(s_1, \dots, s_d)$), tính gradient của $\ln \tilde{p}(\xi \mid y) - \ln q(\xi)$ theo (μ_q, R), cập nhật bằng Adam.
3. ELBO cuối là trung bình của 5% bước cuối (âm của hàm mất mát).
4. Rút 1.000 mẫu $\xi^{(s)} = \mu_q + R\zeta^{(s)}$ từ q cuối, đổi sang $\theta^{(s)} = g(\xi^{(s)})$.

5. Tính trọng số $\ln \tilde{p}(\xi^{(s)} \mid y) - \ln q(\xi^{(s)})$, khớp Pareto tổng quát cho đuôi để được $\hat{k}$.

Xử lý phân kỳ. Tối ưu ngẫu nhiên đôi khi phân kỳ: Tham số biến phân thành NaN sau vài chục bước. Khi đó hàm `vb` không ném lỗi mà trả kết quả có cờ "thất bại" với ELBO $= -\infty$. Bước b08 chạy lại lần khởi tạo đó tối đa hai lần, mỗi lần từ một điểm khởi tạo khác và với tốc độ học giảm một nửa (0,0025 rồi 0,00125). Chỉ khi cả ba lần khởi tạo của một mốc đều thất bại, bước mới dừng và báo rõ mốc và phương pháp.

Gradient tại phiên bị che. Hàm hợp lý tính mật độ cho mọi phiên rồi nhân với mặt nạ; tại phiên bị che, $z_t = 0$. Với GED khi $\nu < 1$, đạo hàm của $|z|^{\nu}$ tại $z = 0$ là vô hạn, và phép che chỉ che giá trị chứ không che đạo hàm, nên gradient thành NaN ($0 \times \infty$). Lỗi này chỉ lộ ra khi GED tham gia lựa chọn và được dùng cho VB. Mã hiện thay z_t tại phiên bị che bằng 1 trước khi tính mật độ (cả EGARCH-X và GARCH), nên giá trị hợp lý không đổi mà gradient luôn hữu hạn.

Với dữ liệu thật, mỗi mốc ước lượng lại chạy VB từ ba điểm khởi tạo (nghiệm MLE của mốc cộng nhiễu $N(0, (0{,}02\,i)^2)$ với $i = 0, 1, 2$) và giữ lần có ELBO lớn nhất; ELBO của cả ba lần được lưu để đánh giá độ ổn định. Chỉ báo $\hat{k}$ tính bằng `arviz.psislw`.

2.7 NUTS

Thuật toán 4. Một chuỗi NUTS có điểm kiểm tra (hàm `nuts_chuoi`)

Đầu vào: Mục tiêu $\ln \tilde{p}(\xi \mid y)$, điểm khởi tạo, 1.000 vòng khởi động, 1.000 mẫu, tỷ lệ chấp nhận đích 0,8, độ sâu cây tối đa 10, ma trận khối lượng đầy đủ.
Đầu ra: Mẫu $\theta^{(s)}$, số bước phân kỳ, số bước nhảy cóc trung bình.

1. Chạy giai đoạn khởi động 1.000 vòng để thích nghi bước ε và ma trận khối lượng. Lưu trạng thái sau khởi động.
2. Chia 1.000 mẫu thành các đoạn 100 vòng. Với mỗi đoạn: Nạp trạng thái sau đoạn trước, chạy 100 vòng, lưu mẫu và trạng thái (gồm ε và ma trận khối lượng đã thích nghi).
3. Nối mọi đoạn thành một chuỗi 1.000 mẫu.

Vì trạng thái lưu gồm cả bước và ma trận khối lượng, chuỗi tiếp tục sau khi dừng là đúng cùng một chuỗi Markov, không phải một chuỗi mới; điều này đã được kiểm thử. Mỗi mốc ước lượng lại chạy 4 chuỗi độc lập (hạt giống khác nhau, khởi tạo $\xi_0 = \hat{\xi}_{\mathrm{MLE}} + N(0, 0{,}05^2 I)$), rồi hàm `chan_doan_nuts` tính $\hat{R}$ chuẩn hóa thứ hạng, cỡ mẫu hiệu dụng phần thân và phần đuôi (qua `arviz`) và số bước phân kỳ. Một mốc "đạt" khi $\hat{R} < 1{,}01$, cỡ mẫu hiệu dụng tối thiểu ≥ 400 và không có phân kỳ.

2.8 Dự báo một phiên

Hàm `du_bao_cam` nhận $\hat{\theta}$ cố định, lọc $\ln h_t$ trên toàn bảng, lấy các phiên trong khoảng dự báo và tính: $\ln h_t$, điểm logarit mật độ, giá trị tích phân xác suất, và với mỗi mức α hai cặp (VaR, ES) của vị thế mua và bán theo (37), với q_α và e_α lấy từ `phan_phoi.py` (công thức đóng cho chuẩn, Student-t, Student-t lệch; cầu phương cho GED). Hàm `du_bao_hau_nghiem` thực hiện hỗn hợp (29): Với mỗi mẫu hậu nghiệm lọc $\ln h_t$ riêng, tính điểm logarit mật độ bằng `logsumexp` trên các mẫu, và lấy VaR, ES từ phân phối gộp của $M = 10.000$ lợi suất mô phỏng.

2.9 Mô phỏng rủi ro nhiều phiên

Thuật toán 5. Mô phỏng VaR và ES H phiên cho một mốc (hàm `mo_phong_moc`)

Đầu vào: Tập tham số P ($|P| = 1$ với MLE, 1.000 mẫu với Bayes), $S = 10.000$ đường, $H_{\max} = 3$, các mức α, chế độ (tham số hoặc lọc).

Đầu ra: Mảng VaR và ES kích thước (số phiên gốc, H, số α, 2 phía).

1. Cho mỗi đường chọn ngẫu nhiên một bộ tham số trong P.
2. Sinh trước ma trận $Z_{s,j}$ ($S \times H_{\max}$): Từ $\mathscr{E}$ với tham số hình dạng của bộ đã chọn, hoặc bốc lại từ các phần dư chuẩn hóa trong mẫu (chế độ lọc). Ma trận này dùng chung cho mọi phiên gốc của mốc.
3. Ước lượng mô hình tự hồi quy bậc một cho từng biến ngoại sinh trên cửa sổ ước lượng; lưu hệ số và phần dư.
4. Với mỗi phiên gốc t trong mốc: Lấy $\ln h_{t+1}^{(s)}$ đã lọc của bộ tham số tương ứng, đặt $X_{t+1}^{(s)}$ bằng giá trị quan sát được, và tích lũy $c^{(s)} \leftarrow 0$.
5. Với $j = 1, \ldots, H_{\max}$:
   (a) $c^{(s)} \leftarrow c^{(s)} + e^{\ln h^{(s)}/2} Z_{s,j}$ và $R^{(s)} \leftarrow e^{c^{(s)}/100} - 1$.
   (b) Với mỗi α và mỗi phía (tổn thất $-R$ hoặc R): Chọn $k = \lceil \alpha S \rceil$ tổn thất lớn nhất bằng `np.partition`; VaR là giá trị nhỏ nhất trong nhóm, ES là trung bình nhóm.
   (c) Nếu $j < H_{\max}$: Cập nhật $X^{(s)} \leftarrow c + \phi X^{(s)} +$ phần dư bốc lại; $s \leftarrow (1 - J)\,[\kappa_1(|Z_{s,j}| - \mu_{|z|}) + \kappa_2 Z_{s,j}]$; $\ln h^{(s)} \leftarrow \mathrm{clip}(\omega + s + \beta \ln h^{(s)} + \gamma^\top X^{(s)} + \delta^\top D, -20, 20)$.

Hai đặc điểm của cài đặt cần nêu. Thứ nhất, các số ngẫu nhiên $Z_{s,j}$ dùng chung giữa các phiên gốc của một mốc (số ngẫu nhiên chung): Điều này làm các dự báo kề nhau mượt hơn và giảm khối lượng tính toán, nhưng làm sai số Monte Carlo của các phiên kề nhau tương quan. Thứ hai, mô phỏng đặt $\mu = 0$ cho lợi suất tích lũy, một lựa chọn thận trọng cho ký quỹ.

2.10 Tổn thất thực tế và các thước đo đối chứng

Hàm `tong_that_thuc` tính $R_{t,H} = \exp\left(\sum_{j=1}^{H} r_{t+j}/100\right) - 1$ cho $H = 1,2,3$ từ chuỗi lợi suất cùng hợp đồng (đã lấp tại phiên chuyển bằng giá F2M), và cờ hợp lệ (cửa sổ $(t, t+H]$ không chứa lợi suất khuyết). Nhờ vậy tổn thất qua phiên chuyển là tổn thất của một vị thế được cuộn sang hợp đồng mới. Mô phỏng lịch sử `san_lich_su` lấy VaR H phiên là phân vị của các $R_{s,H}$ đã hoàn tất trước t (cửa sổ mở rộng, cần tối thiểu 100 quan sát) và chọn phía lớn hơn; `lich_su_cua_so` làm tương tự với cửa sổ 250 phiên; `ewma_var` cho VaR chuẩn $z_{1-\alpha}\sqrt{H\hat{\sigma}_{t+1}^2}/100$ với $\hat{\sigma}^2$ làm trơn mũ hệ số 0,94.

2.11 Quy tắc ký quỹ năm bước

Thuật toán 6. Quy tắc ký quỹ (hàm `quy_tac`)

Đầu vào: Chuỗi $\widehat{\mathrm{VaR}}_t$, $\widehat{\mathrm{ES}}_t$, sàn lịch sử V_t^{LS} (cùng độ dài, theo thời gian); (w, b, φ, η); $\Delta = 0{,}01$, $g = 0{,}005$.

Đầu ra: Chuỗi m_t.

1. $m_t^{(1)} \leftarrow (1-w)\widehat{\mathrm{VaR}}_t + w\widehat{\mathrm{ES}}_t$, rồi $m_t^{(3)} \leftarrow \max\{(1+b)\,m_t^{(1)},\ \varphi V_t^{LS}\}$.
2. Với phiên hợp lệ đầu tiên: $m^{(4)} \leftarrow m_t^{(3)}$ và $m \leftarrow g\lceil m_t^{(3)}/g\rceil$.
3. Với mỗi phiên hợp lệ tiếp theo:
   (a) $m_t^{(4)} \leftarrow \max\{m_t^{(3)},\ \eta\, m_{t-1}^{(4)} + (1-\eta)\, m_t^{(3)}\}$.
   (b) $m_t \leftarrow \max\left\{\varphi V_t^{LS},\ g\lceil \max(m_t^{(4)},\ m_{t-1} - \Delta)/g\rceil\right\}$.

Phép làm tròn lên có trừ một sai số số học nhỏ (10^{-12}) để tránh làm tròn lệch một bậc do sai số dấu phẩy động khi tỷ số là số nguyên. Các tham số $g = 0{,}5\%$, $\Delta = 1\%$ và mức ký quỹ đối chứng 17% được đọc từ `the_che.toml`, nơi mỗi thông số phải có nguồn và ngày xác minh; hồ sơ chính thức không chạy nếu còn thông số thiếu nguồn.

2.12 Chỉ tiêu và kiểm định

Hàm `chi_tieu` tính tỷ lệ vượt, ký quỹ trung bình và lớn nhất, thiếu hụt bình quân và lớn nhất, độ lệch chuẩn của biến động ký quỹ, số lần điều chỉnh và tỷ số thuận chu kỳ $\max_t m_{t+20}/m_t$ trên các phiên hợp lệ. Hàm `kiem_dinh_vuot` chạy Kupiec và Christoffersen trên dãy lấy thưa mỗi H phiên và kiểm định phân vị động với phương sai vững (HAC) trên toàn dãy chồng lấn. Chi phí kỳ vọng và đường biên đánh đổi do `chi_phi` và `duong_bien` tính. Trong `danh_gia.py`: `kupiec`, `christoffersen`, `phan_vi_dong`, `fz0`, `dm` (Diebold–Mariano với phương sai Newey–West), `giacomini_white`, `spa` và `mcs` (bootstrap dừng, độ dài khối 5, 2.000 lần lặp), `berkowitz`, `vif` và `den_giao_thong`.

3 Hạ tầng bảo đảm tính đúng đắn

Nghiên cứu cần chạy liên tục nhiều ngày trên một máy, nên hạ tầng phải bảo đảm ba điều: Dừng và chạy tiếp không làm thay đổi kết quả, kết quả cũ không bị dùng lại khi đầu vào đã đổi, và khối kiểm tra không rò rỉ. Các cơ chế sau thực hiện việc này.

3.1 Bước, đơn vị và hồ sơ

Mỗi bước (đối tượng `Buoc`) khai báo một danh sách *đơn vị việc* dạng `nhóm::khóa`. Các nhóm độc lập nên chạy song song; các khóa trong cùng một nhóm chạy tuần tự để có thể khởi tạo ấm từ đơn vị trước. Chẳng hạn, đơn vị `mle|skewt|G::055` là MLE cấu hình "skew-t, biến G" tại mốc thứ 56. Mỗi bước có một hồ sơ ghi trạng thái (đang chạy, xong, cổng không đạt), mã băm đầu vào, mã băm đầu ra và thời điểm.

3.2 Mã băm đầu vào

Mã băm đầu vào của một bước gộp: Cấu hình các mục mà bước khai báo, mã băm đầu ra của các bước mà nó phụ thuộc, nội dung các tệp mã nguồn mà nó dùng, và nội dung dữ liệu gốc. Thư mục điểm kiểm tra có tên là mười hai ký tự đầu của mã băm này. Hệ quả: (i) Nếu đầu vào đổi, thư mục điểm kiểm tra mới trống và bước chạy lại; (ii) hai máy chạy hai phiên bản mã khác nhau ghi vào hai thư mục khác nhau, không thể trộn lẫn; (iii) những phần cấu hình không ảnh hưởng con số (số tiến trình, bộ nhớ, phân mảnh) không nằm trong mã băm, nên đổi chúng không làm chạy lại.

Hai ngoại lệ có chủ đích được cài đặt. Thứ nhất, cờ `mo_rong_don_vi` (dùng cho mô phỏng khôi phục b03a và b03b) cho phép tăng số lần lặp hoặc thêm phân phối mà chỉ chạy thêm các đơn vị còn thiếu, vì kết quả mỗi lần lặp chỉ phụ thuộc hạt giống riêng. Danh sách đơn vị của hai bước này được đặt trong các tệp riêng `b03b_so_lan_lap.py` và `b03a_ged.py`, không nằm trong tệp được băm `b03_mo_phong.py`; nhờ vậy việc thêm GED vào mô phỏng khôi phục (Mục 5) không làm mất các đơn vị đã xong. Thứ hai, các bước chỉ dùng khối UL (dữ liệu, kiểm tra cài đặt) không băm cỡ khối XT, vì kết quả của chúng không phụ thuộc cỡ này.

3.3 Điểm kiểm tra ba tầng

Tầng 1 (cả bước). Hồ sơ bước: Nếu đã xong với cùng mã băm đầu vào thì bước bị bỏ qua.

Tầng 2 (từng đơn vị). Mỗi đơn vị xong được ghi thành một tệp riêng; khi chạy lại, các đơn vị đã có tệp được nạp thay vì tính lại.

Tầng 3 (trong đơn vị). Với các đơn vị dài, trạng thái giữa chừng được lưu: Chuỗi NUTS sau khởi động và sau mỗi 100 vòng lấy mẫu; VB sau mỗi 2.000 bước tối ưu (gồm trạng thái bộ tối ưu Adam và lịch sử hàm mất mát).

Mọi tệp được ghi nguyên tử: Ghi vào tệp tạm cùng thư mục rồi đổi tên, nên không bao giờ có tệp ghi dở. Trên Windows, khi phần mềm diệt virus hoặc dịch vụ đồng bộ giữ tệp, thao tác đổi tên được thử lại tối đa 10 lần.

3.4 Tính tất định và hạt giống

Hạt giống của mỗi đơn vị là băm của (hạt giống chung, mã bước, khóa đơn vị). Vì vậy làm lại một đơn vị ở bất kỳ thời điểm nào cũng cho cùng kết quả. Dữ liệu mô phỏng của mỗi lần lặp (phân phối, trường hợp, chỉ số) có hạt giống riêng không phụ thuộc bước, nên mô phỏng MLE và mô phỏng Bayes được đánh giá trên cùng các bộ dữ liệu giả.

3.5 Phân mảnh và song song hóa

Trên một máy, các nhóm đơn vị được chia cho các tiến trình con; số tiến trình mặc định là số nhỏ hơn giữa (số lõi -1) và (RAM trống chia 1,5 GB), và giảm một nửa sau mỗi lần một tiến trình con bị hệ điều hành dừng vì thiếu bộ nhớ. Mỗi tiến trình dùng một luồng để tránh tranh chấp lõi giữa JAX và thư viện đại số tuyến tính. Mỗi nhóm có đúng một máy sở hữu, xác định tĩnh bằng băm tên nhóm trên danh sách phân mảnh trong cấu hình; thiết kế này cho phép thêm máy mà không đổi mã.

3.6 Cổng chuyển giai đoạn và khóa khối kiểm tra

Bước b07 kiểm tra ba điều kiện của Mục 8 (Tài liệu 1). Nếu không đạt, bước ghi trạng thái "cổng không đạt", in lý do và dừng toàn bộ chuỗi (mã thoát 3); chỉ khi người dùng chủ động chạy lại với cờ chấp nhận, bước mới tiếp tục và việc chấp nhận được ghi vào hồ sơ cùng phụ lục sai lệch của báo cáo. Khi cổng đạt, bước ghi tệp `KHOA_CAU_HINH.json` (phân phối, biến, ranh giới khối).

Trước khi có tệp khóa, hàm cấp dữ liệu cho mọi bước cắt bảng ngày tại hết khối XT, nên không có dòng mã nào có thể đọc khối KT; bước khai báo cần khối KT sẽ bị từ chối. Bước kiểm toán b15 kiểm tra thêm rằng không có đầu ra nào của giai đoạn 1 chứa quan sát thuộc KT.

3.7 Đồng bộ và vòng lặp điều phối

Điểm vào `chay.py --buoc tat_ca` chạy chuỗi chính theo thứ tự các bước, mỗi bước trong một hộp thời gian 30 phút; hết hộp thì đẩy kết quả lên kho git, kéo thay đổi mới, rồi tiếp tục từ điểm kiểm tra. Nếu mã nguồn, cấu hình hoặc dữ liệu đầu vào (`du_lieu/goc`, `du_lieu/ngoai`, trừ tệp ghi chú) trên kho đã đổi so với mã băm lúc khởi động, tiến trình dừng sạch với mã thoát 4 và tệp `chay.bat` khởi động lại bằng bản mới. Dữ liệu đầu vào chỉ được đưa vào mã băm này từ ngày 23/10/2026: Trước đó, việc bổ sung giá F2M ngày 15/08/2019 không làm máy khởi động lại, nên một lần chạy đã tiếp tục trên dữ liệu cũ (Tài liệu 3). Khi cổng b07 không đạt (mã thoát 3),

`chay.bat` dừng hẳn và phải được mở lại bằng tay. Khi chuỗi chính phải chờ hoặc đã xong, mô phỏng Bayes (b03b) chạy ở nền. Sau khi mô phỏng Bayes xong, các bước báo cáo, kiểm toán và đóng gói được chạy lại để báo cáo có đủ mục mô phỏng.

4 Kiểm thử và kiểm chứng cài đặt

Bảng 3: Các lớp kiểm chứng

Đối tượng	**Phép kiểm tra**
Phân phối	Mật độ tích phân bằng 1, phương sai bằng 1; $\mu_{\lvert z\rvert}$ của bản JAX khớp cầu phương; trường hợp giới hạn ($\nu \to \infty$, $\lambda = 0$) trùng phân phối đơn giản hơn; VaR, ES khớp Bảng 7 (Tài liệu 1) đến 3 chữ số
Hàm hợp lý	Bản JAX khớp bản numba độc lập (log-hợp lý chênh $\sim 10^{-12}$, MLE chênh $\sim 10^{-8}$, sai số chuẩn trùng)
Đối chiếu gói ngoài	EGARCH khớp gói `arch`; GARCH(1,1) khớp gói `arch`; MCS và SPA trùng kết luận
Mật độ hậu nghiệm	Số hạng Jacobian và tiên nghiệm hữu hạn tại biên
NUTS	Dừng sau khởi động rồi chạy tiếp cho đúng số mẫu
Điểm kiểm tra	Mất một tệp thì chỉ tính lại đúng đơn vị đó; kết quả tổng hợp giống hệt; bước "mở rộng đơn vị" chỉ chạy phần thiếu
Khóa khối kiểm tra	Dữ liệu bị cắt tại hết khối XT; bước cần khóa bị từ chối
Phân mảnh	Mỗi nhóm có đúng một chủ, phủ đủ, tất định

Ngoài kiểm thử đơn vị, đường ống được chạy thử trọn vẹn từ b00 đến b15 ở hồ sơ rút gọn (ít vòng lặp, ít mốc) mỗi khi đổi mã, để xác nhận các bước nối với nhau đúng trước khi chạy hồ sơ đầy đủ.

5 Những điểm cần lưu ý khi diễn giải

1. **Biến phân dùng một mẫu mỗi bước.** Gradient của ELBO ước lượng bằng một mẫu ζ mỗi bước, tốc độ học không giảm dần, và tham số cuối là của bước cuối chứ không phải trung bình các bước cuối. Cách làm này phổ biến nhưng có thể để lại dao động nhỏ quanh cực trị; độ ổn định qua ba điểm khởi tạo được dùng để kiểm soát.
2. **Số ngẫu nhiên chung trong mô phỏng nhiều phiên.** Như đã nêu ở Mục 2, sai số Monte Carlo của VaR, ES các phiên kề nhau tương quan. Điều này không làm chệch dự báo nhưng ảnh hưởng đến độ chính xác của kiểm định trên chuỗi dự báo.
3. **Quy đổi ký quỹ ra tiền chưa có trong đầu ra.** Mọi chỉ tiêu ký quỹ được tính theo tỷ lệ giá trị hợp đồng; phép nhân với hệ số nhân và giá hợp đồng (Tài liệu 1, Mục 7) chưa được đưa vào báo cáo tự động.

4. **GARCH nền không có biến lịch.** GARCH(1,1) được cài đặt theo dạng chuẩn, không có biến giả lịch; EGARCH (khi $\gamma = 0$) giữ biến lịch. Sự khác biệt này nên được nêu khi diễn giải kết quả so sánh GARCH với EGARCH.

5. **Chẩn đoán hồi quy theo biến lịch bỏ biến phiên chuyển.** Trong chẩn đoán phần dư, hồi quy $\hat{z}_t^2 - 1$ lên biến lịch chỉ dùng biến đáo hạn và biến nghỉ dài, vì trước đây phiên chuyển không có lợi suất. Từ khi dùng giá F2M, phiên chuyển có lợi suất, nên hồi quy này nên thêm biến phiên chuyển. Chẩn đoán này không thuộc điều kiện của cổng (điều kiện (ii) dùng hồi quy theo nhóm τ_t). Việc sửa vẫn được hoãn: Sửa `danh_gia.py` làm đầu ra của b02 đổi, kéo theo b03a (16.000 đơn vị) chạy lại từ đầu.

6. **GED trong mô phỏng khôi phục (từ 23/10/2026).** Ban đầu mô phỏng khôi phục chỉ chạy cho chuẩn, Student-t và Student-t lệch. Khi bước chọn phân phối chọn GED, cổng b07 không đạt vì điều kiện (i) không có số liệu cho GED. GED được bổ sung vào cả hai bước: b03a thêm 4.000 đơn vị MLE (tổng 16.000), b03b thêm 29.800 đơn vị VB, NUTS và SBC (tổng 119.200). Tham số thật của GED trong mô phỏng là $\nu = 1{,}5$; các tham số còn lại như ba phân phối kia.

7. **Mã băm và đổi mã.** Mọi sửa đổi mã nguồn của một bước hay của mô-đun mà nó dùng đều làm bước chạy lại. Cơ chế này an toàn nhưng cần được tính đến khi đổi mã giữa chừng một lần chạy dài.